

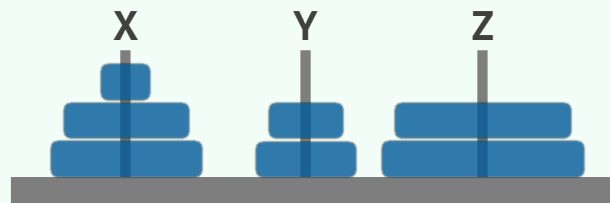
栈

栈接口与实现

操作与接口

❖ 栈 (stack) 是受限的序列

- 只能在栈顶 (top) 插入和删除
- 栈底 (bottom) 为盲端



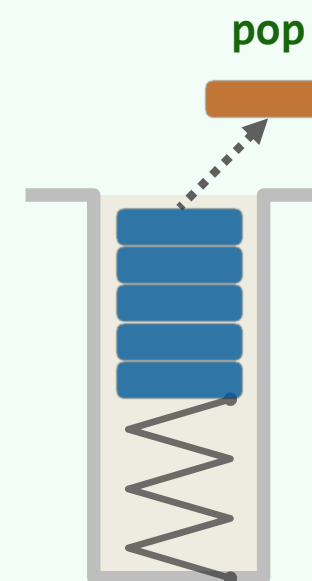
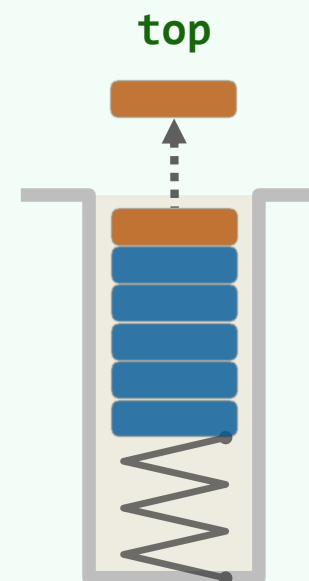
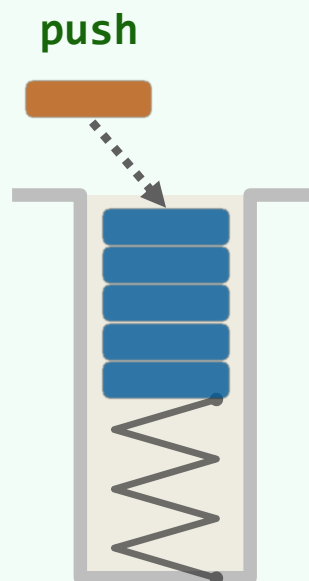
❖ 基本接口

- size() / empty()
- push() 入栈
- pop() 出栈
- top() 查顶

❖ 后进先出 (LIFO)

先进后出 (FILO)

❖ 扩展接口: getMax()...



实例

操作	输出	栈 (左侧栈顶)
Stack()		
empty()	true	
push(5)		5
push(3)		3 5
pop()	3	5
push(7)		7 5
push(3)		3 7 5
top()	3	3 7 5
empty()	false	3 7 5

操作	输出	栈 (左侧栈顶)
push(11)		11 3 7 5
size()	4	11 3 7 5
push(6)		6 11 3 7 5
empty()	false	6 11 3 7 5
push(7)		7 6 11 3 7 5
pop()	7	6 11 3 7 5
pop()	6	11 3 7 5
top()	11	11 3 7 5
size()	4	11 3 7 5

实现

❖ 栈既然属于序列的特例，故可直接基于向量或列表**派生**

❖ `template <typename T> class Stack: public Vector<T> {`

`public: //原有接口一概沿用`

`void push( T const & e ) { insert( e ); } //入栈`

`T pop() { return remove( size() - 1 ); } //出栈`

`T & top() { return (*this)[ size() - 1 ]; } //取顶`

`};` **//以向量首/末端为栈底/顶——颠倒过来呢?**

换成列表呢?

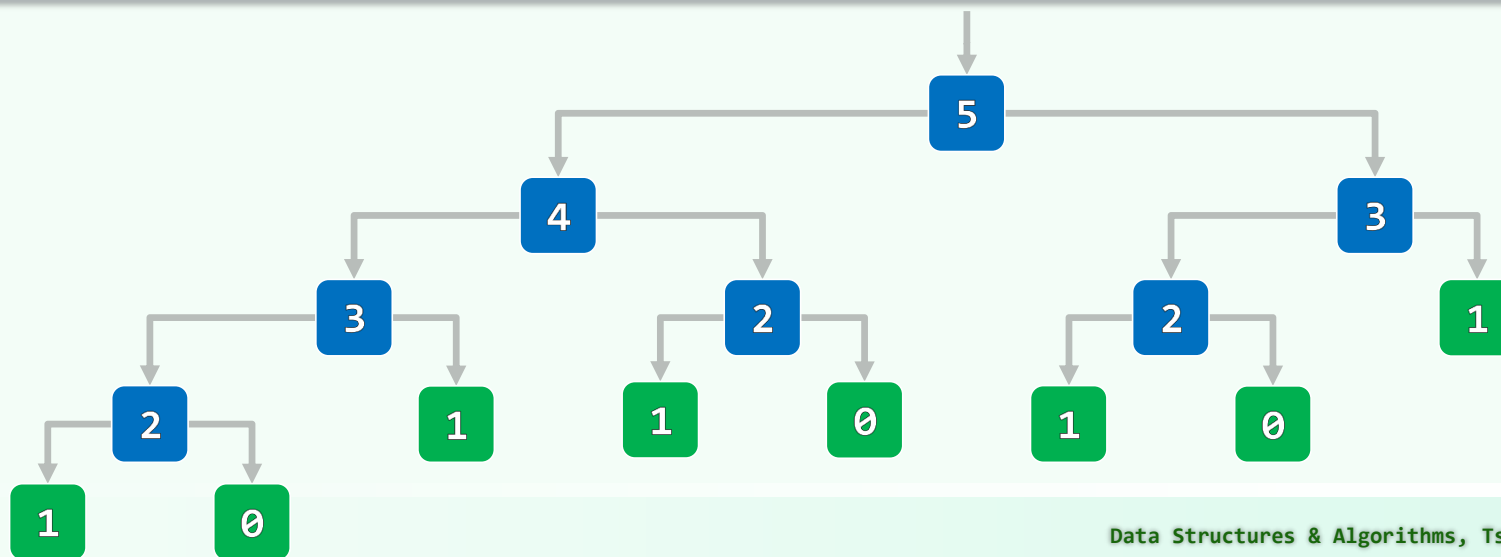
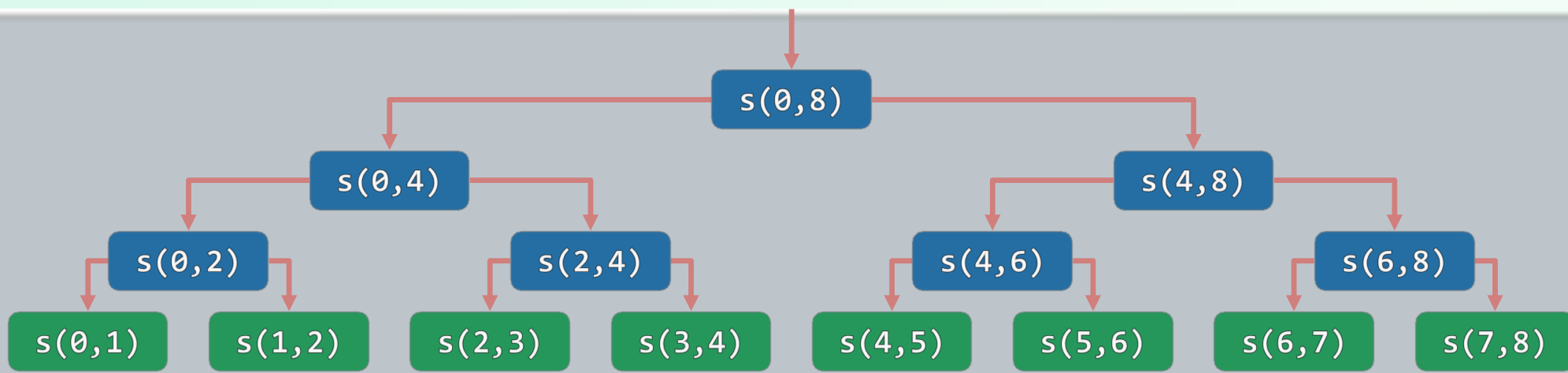
❖ 确认：如此实现的栈各接口，均只需 $O(1)$ 时间

❖ 课后：基于**列表**，派生定义**栈模板类**；你所实现的栈接口，效率如何？

栈

调用栈：原理与算法

函数调用树：如何实现？Theseus的线团 + 粉笔



call stack

xxxx9000

main()

i = 9, ...

binary executable

```
xxxx0700: main() {  
           int i = 9;  
           /* ... */  
xxxx1000:   funcA(i * i);  
           /* ... */  
           }  
xxxx1200: void funcA(int j) {  
           int k = j / 3;  
           /* ... */  
xxxx1500:   funcB(k + 5);  
           /* ... */  
           }  
xxxx1750: void funcB(int m) {  
           int i = m / 4;  
           /* ... */  
xxxx1820:   funcB(m - i);  
           /* ... */  
           }
```

call stack

xxxx9000

main()

i = 9, ...

xxxx8850

funcA()

return address = 1000

j = 81, k = 27, ...

previous frame = 9000

binary executable

xxxx0700: main() {

int i = 9;

/* ... */

xxxx1000: funcA(i * i);

/* ... */

}

xxxx1200: void funcA(int j) {

int k = j / 3;

/* ... */

xxxx1500: funcB(k + 5);

/* ... */

}

xxxx1750: void funcB(int m) {

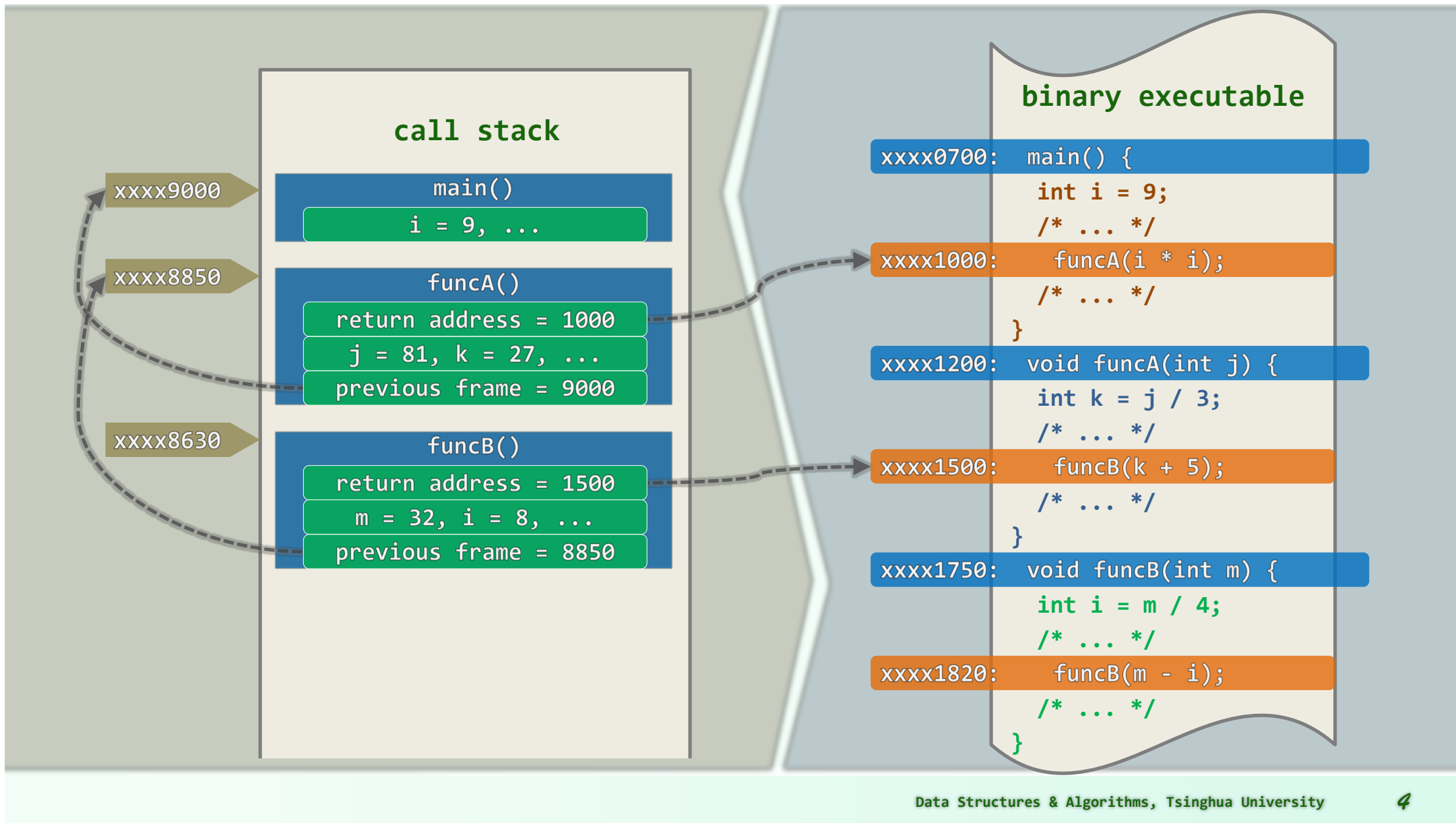
int i = m / 4;

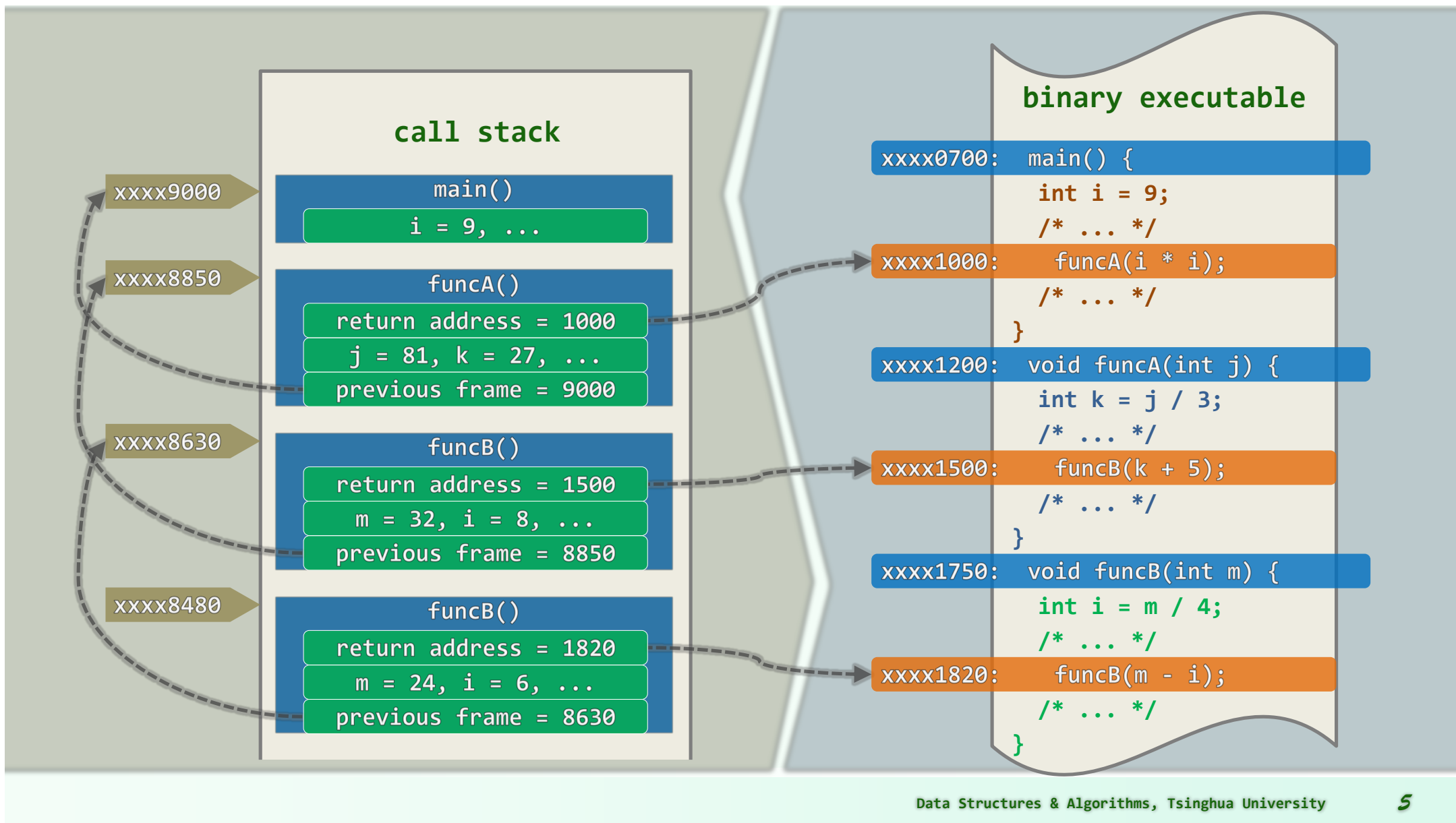
/* ... */

xxxx1820: funcB(m - i);

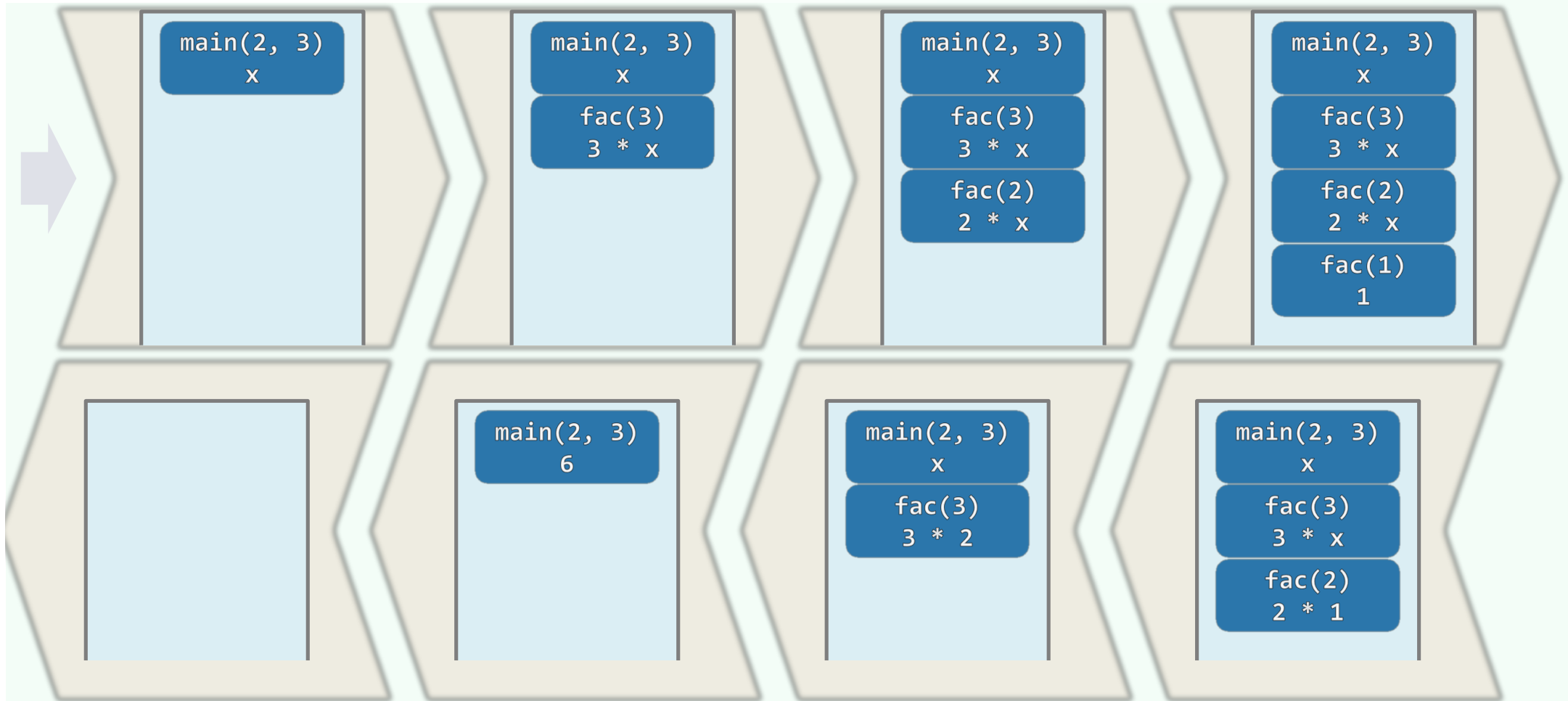
/* ... */

}

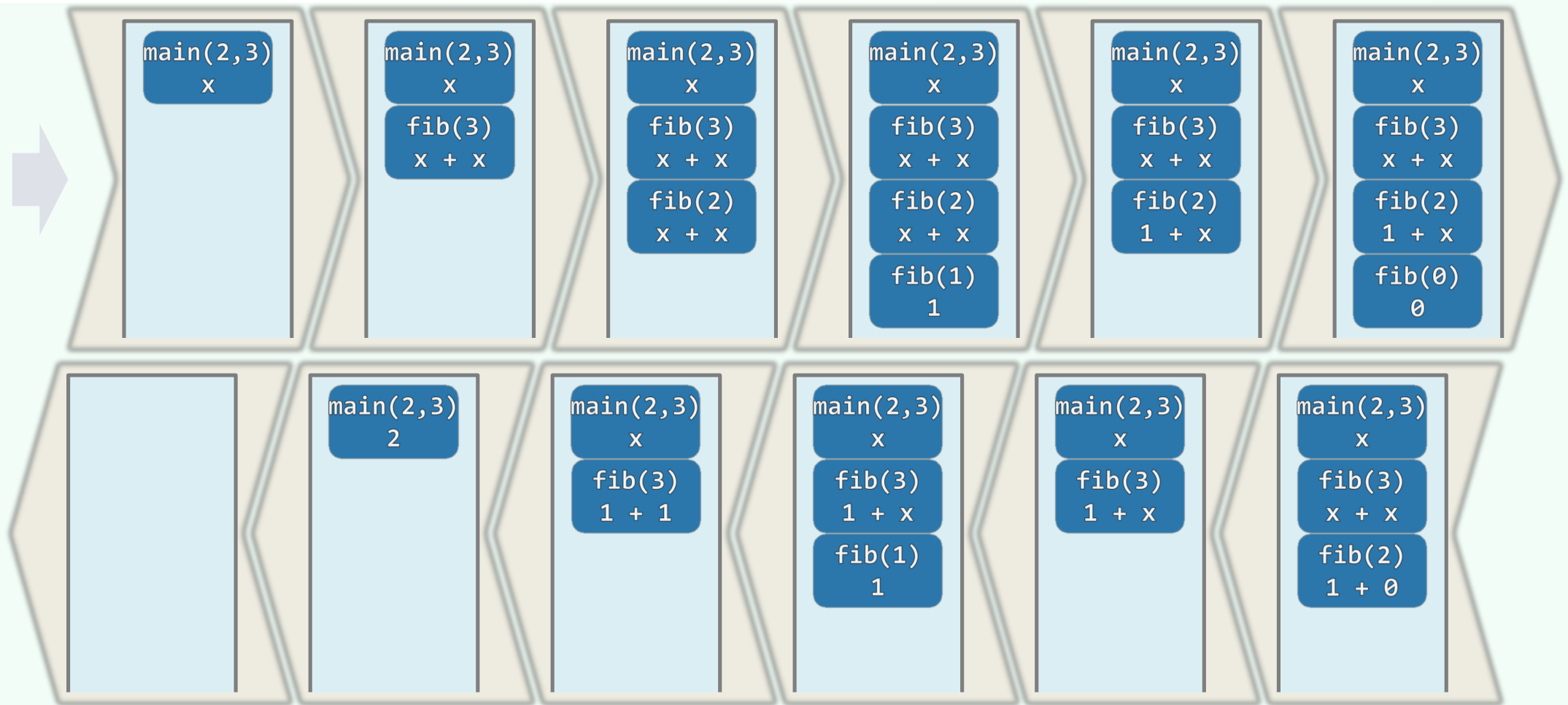




```
int fac(int n) { return (n < 2) ? 1 : n * fac(n - 1); }
```



```
int fib( int n ) { return (n < 2) ? n : fib(n - 1) + fib(n - 2); }
```



栈 进制转换

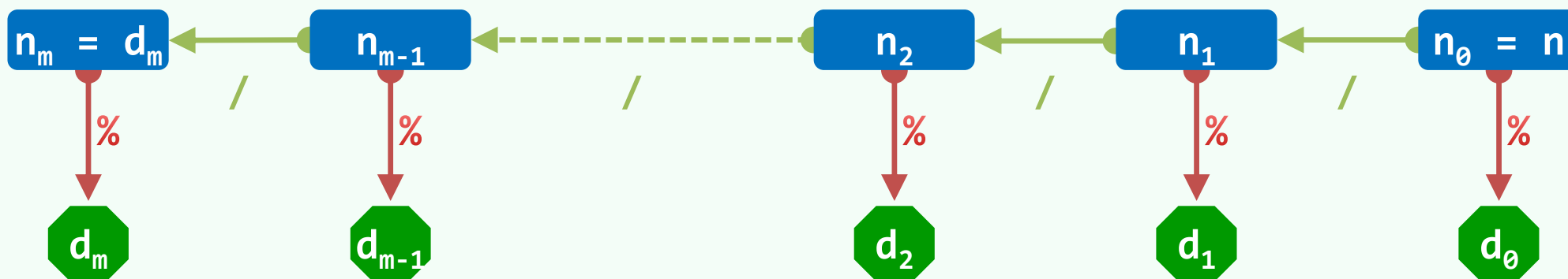
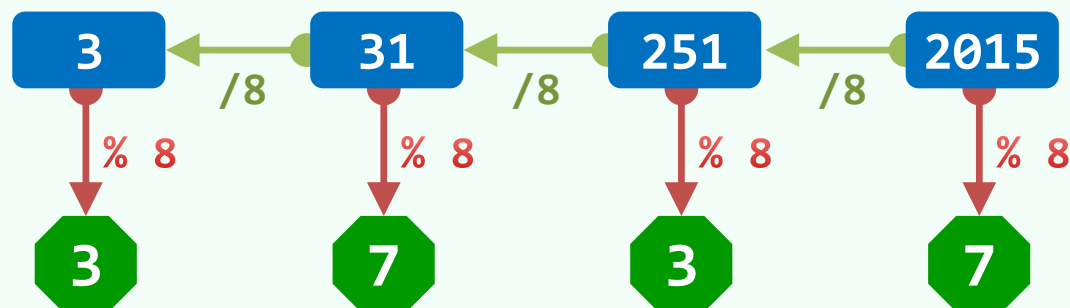
短除法：整商 + 余数

❖ 设： $n = (d_m \dots d_2 d_1 d_0)_\lambda = d_m \cdot \lambda^m + \dots + d_2 \cdot \lambda^2 + d_1 \cdot \lambda^1 + d_0 \cdot \lambda^0$

令： $n_i = (d_m \dots d_{i+2} d_{i+1} d_i)_\lambda$

❖ 则有： $n_{i+1} = n_i / \lambda$ 和 $d_i = n_i \% \lambda$

❖ 如此对 λ 反复整除、留余，即可自低而高得出 λ 进制的各位



实现

```
❖ void convert( Stack<char> & S, __int64 n, int base ) {  
    char digit[] = "0123456789ABCDEF"; //数位符号，如有必要可相应扩充  
    while ( n > 0 ) { //由低到高，逐一计算出新进制下的各数位  
        S.push( digit[ n % base ] ); //余数（当前的数位）入栈  
        n /= base; //n更新为其对base的除商  
    }  
} //新进制下由高到低的各数位，自顶而下保存于栈s中  
  
❖ main() {  
    Stack<char> S; convert( S, n, base ); //用栈记录转换得到的各数位  
    while ( ! S.empty() ) printf( "%c", S.pop() ); //逆序输出  
}
```

栈 括号匹配

实例

❖ $(a[i-1][j+1]) + a[i+1][j-1]) * 2$

//失配

$(a[i-1][j+1] + a[i+1][j-1]) * 2$

//匹配

❖ 观察：除了各种括号，其余符号均可暂时忽略

$([] []) [] [])$

//失配

$([] [] [] [])$

//匹配

❖ 从简单入手，先来考查只有一种括号的情况...

尝试：由外而内

0) 平凡： 无括号的表达式是匹配的

1) 减治？ E 匹配，仅当 (E) 匹配

2) 分治？ E 和 F 均匹配，仅当 E F 匹配

❖ 然而，根据以上性质，却不易直接应用已知的策略

❖ 究其根源在于，1) 和2) 均为**充分性**，比如反例：

(() ()) () = ((() ()) ())

(() ()) () = (()) (()) ()

❖ 而为使问题有效简化，必须发现并借助**必要性**

构思：由内而外

❖ 颠倒以上思路：消去一对**紧邻**的左右括号，不影响全局的匹配判断

亦即：

L

()

R

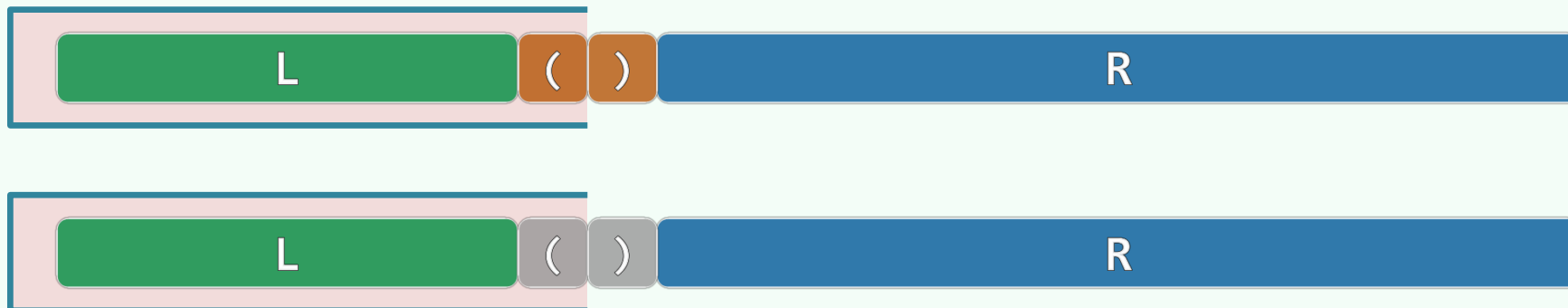
 匹配，仅当

L

R

 匹配

❖ 那么，如何**找到**这对括号？再者，如何使问题的这种简化得以**持续**进行？



❖ 顺序扫描表达式，用栈记录已扫描的部分

//实际上只需记录左括号

反复迭代：凡遇"("，则进栈；凡遇")"，则出栈

实现

```
bool paren( const char exp[], int lo, int hi ) { //exp[lo, hi)

    Stack<char> S; //使用栈记录已发现但尚未匹配的左括号

    for ( int i = lo; i < hi; i++ ) //逐一检查当前字符

        if ( '(' == exp[i] ) S.push( exp[i] ); //遇左括号: 则进栈

        else if ( ! S.empty() ) S.pop(); //遇右括号: 若栈非空, 则弹出左括号

        else return false; //否则 (遇右括号时栈已空) , 必不匹配

    return S.empty(); //最终栈空, 当且仅当匹配

}
```


实例：一种括号

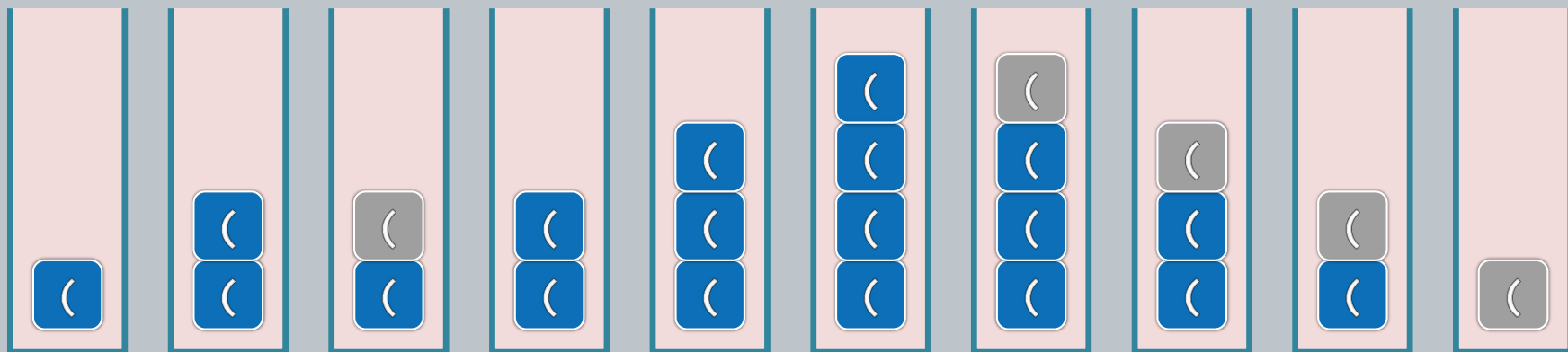
❖ 实际上，若仅考虑一种括号，只需一个计数器足矣

`//S.size()`

❖ 一旦转负，则为失配（右括号多余）；最后归零，即为匹配（否则左括号多余）

1 2 1 2 3 4 3 2 1 0

(() ((())))

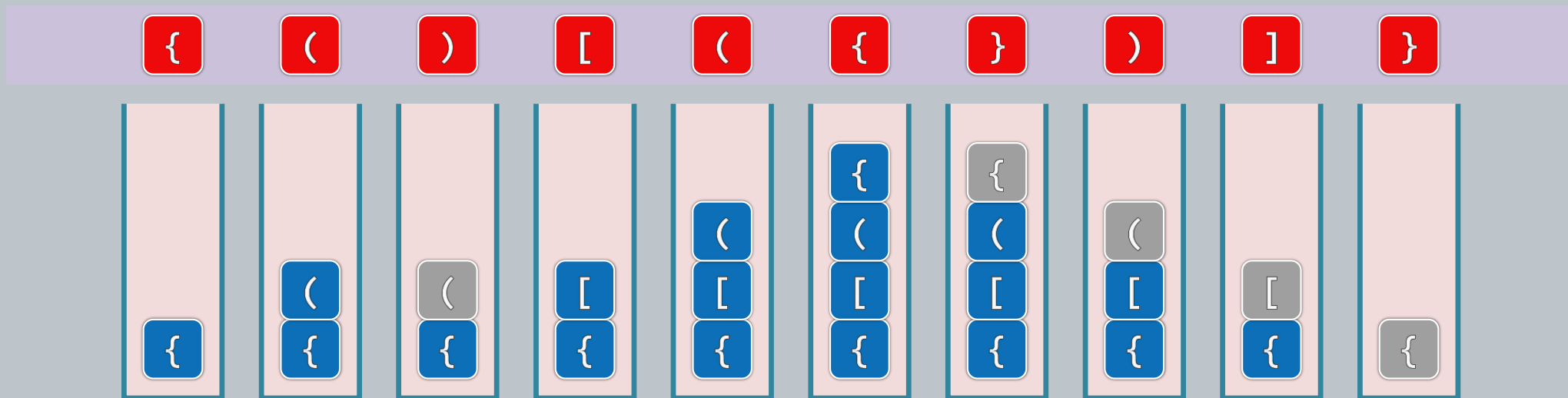


拓展：多类括号

❖ 以上思路及算法，可否推广至多种括号并存的情况？反例： `[ ( ] )`

❖ 甚至，只需约定“括号”的通用格式，而不必事先固定括号的类型与数目

比如：`<body>|</body>`，`<h1>|</h1>`，`<font>|</font>`，`<p>|</p>`，`<ol>|</ol>`，...



栈
栈混洗

Stack Permutation

❖ 考查栈 $\mathcal{A} = \langle a_1, a_2, a_3, \dots, a_n \rangle$

$$\mathcal{B} = \mathcal{S} = \emptyset$$

❖ 只允许

- 将 $\mathcal{A}$ 的顶元素弹出并压入 $\mathcal{S}$, 或
- 将 $\mathcal{S}$ 的顶元素弹出并压入 $\mathcal{B}$

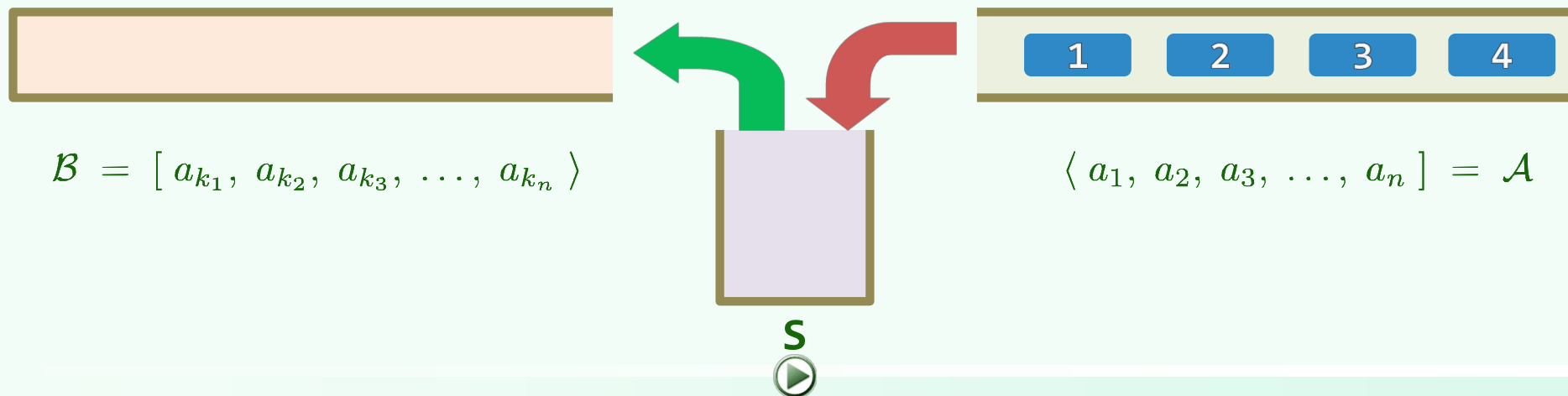
❖ 亦即 $\mathcal{S}.push(\mathcal{A}.pop())$

$\mathcal{B}.push(\mathcal{S}.pop())$

❖ 若经一系列以上操作后, $\mathcal{A}$ 中元素全部转入 $\mathcal{B}$ 中

$$\mathcal{B} = \langle a_{k_1}, a_{k_2}, a_{k_3}, \dots, a_{k_n} \rangle$$

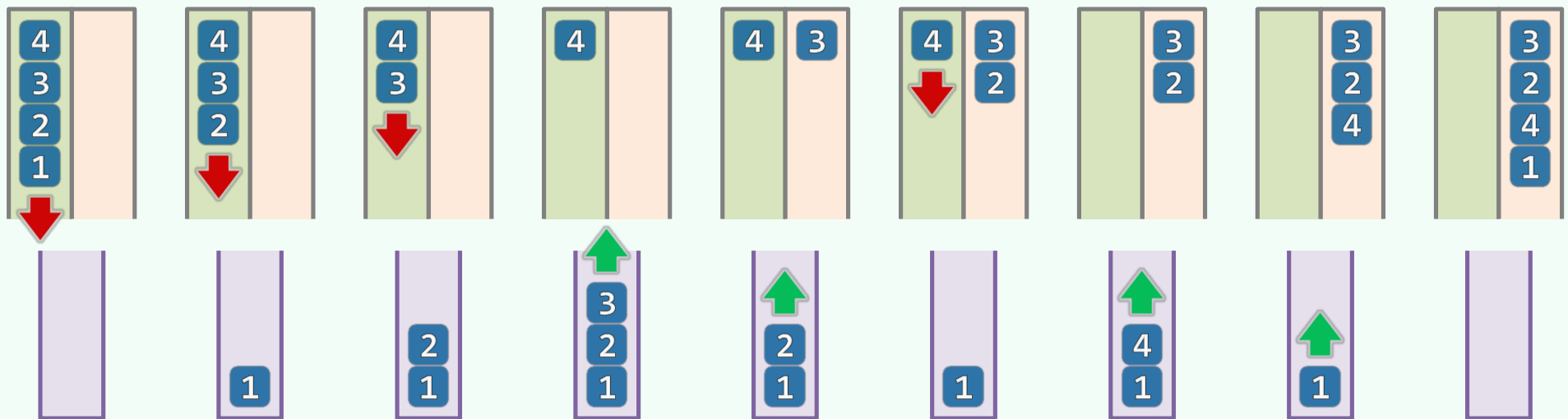
则称为 $\mathcal{A}$ 的一个**栈混洗**



计数: $SP(n)$

❖ 同一输入序列, 可有多种栈混洗: $[1, 2, 3, 4 >$, $[4, 3, 2, 1 >$, $[3, 2, 4, 1 > \dots$

❖ 一般地, 对于长度为 n 的序列, 混洗总数 $SP(n) = ?$



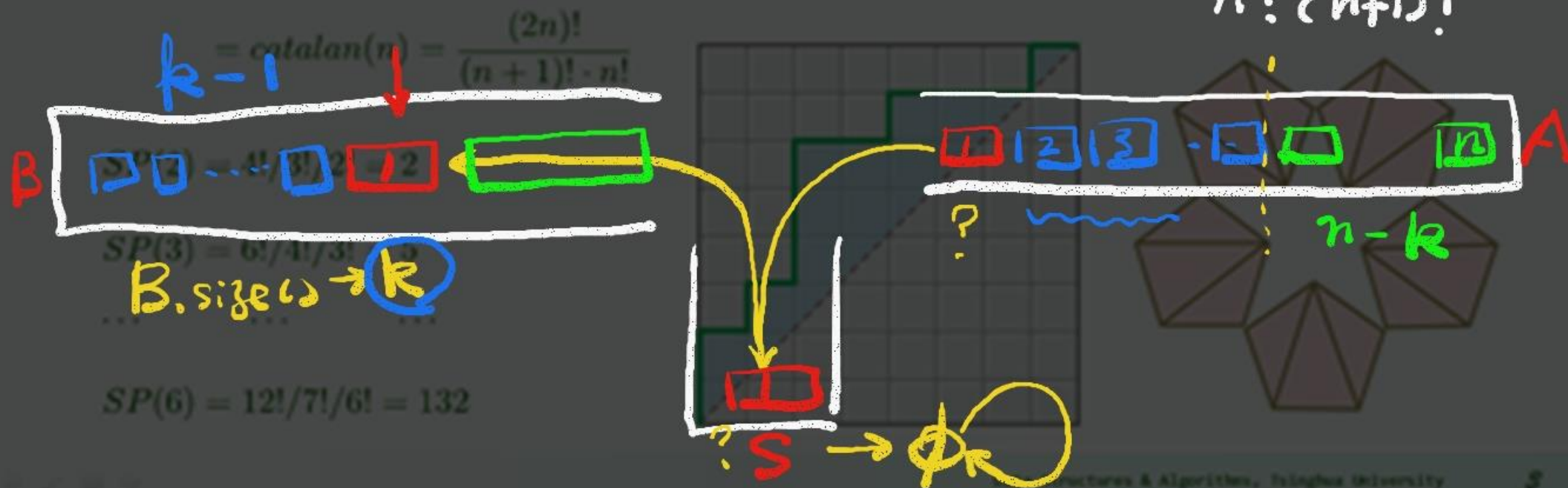
❖ 显然, $SP(n) \leq n!$; 更准确地呢?

计数: catalan(n)

$$SP(n) = \sum_{k=1}^n SP(k-1) \cdot SP(n-k)$$

✧ 考查S再度变空 (A首元素从S中弹出的时刻, 无非n种情况:

$$SP(n) = \sum_{k=1}^n SP(k-1) \cdot SP(n-k) = \text{catalan}(n) = \frac{(2n)!}{n!(n+1)!}$$



甄别：检测禁形

❖ 输入序列 $\langle 1, 2, 3, \dots, n \rangle$ 的任一排列 $[p_1, p_2, p_3, \dots, p_n]$ 是否为栈混洗？

❖ 先考查简单情况： $n = 3, A = \langle 1, 2, 3 \rangle$

- 栈混洗共 $6! / 4! / 3! = 5$ 种；全排列共 $3! = 6$ 种 //少了一种...

❖ $[3, 1, 2]$ //为什么是它？

❖ 观察：任意三个元素能否按某相对次序出现于混洗中，与其它元素无关 //故可推而广之...

❖ 禁形：对于任何 $1 \leq i < j < k \leq n$

$[\dots, \boxed{k}, \dots, \boxed{i}, \dots, \boxed{j}, \dots]$ 必非栈混洗

❖ 反过来，不存在 “ $\boxed{312}$ ”模式的序列，一定是栈混洗吗？

甄别：直接模拟

❖ 充要性： A permutation is a stack permutation **iff**

(Knuth, 1968) it does NOT involve the permutation **312** //习题[4-3]

❖ 如此，可得一个 $O(n^3)$ 的甄别算法 //进一步地...

❖ $[p_1, p_2, p_3, \dots, p_n]$ 是 $[1, 2, 3, \dots, n]$ 的栈混洗，**当且仅当**

对于任意 $i < j$ ，不含模式 $[\dots, j+1, \dots, i, \dots, j, \dots]$

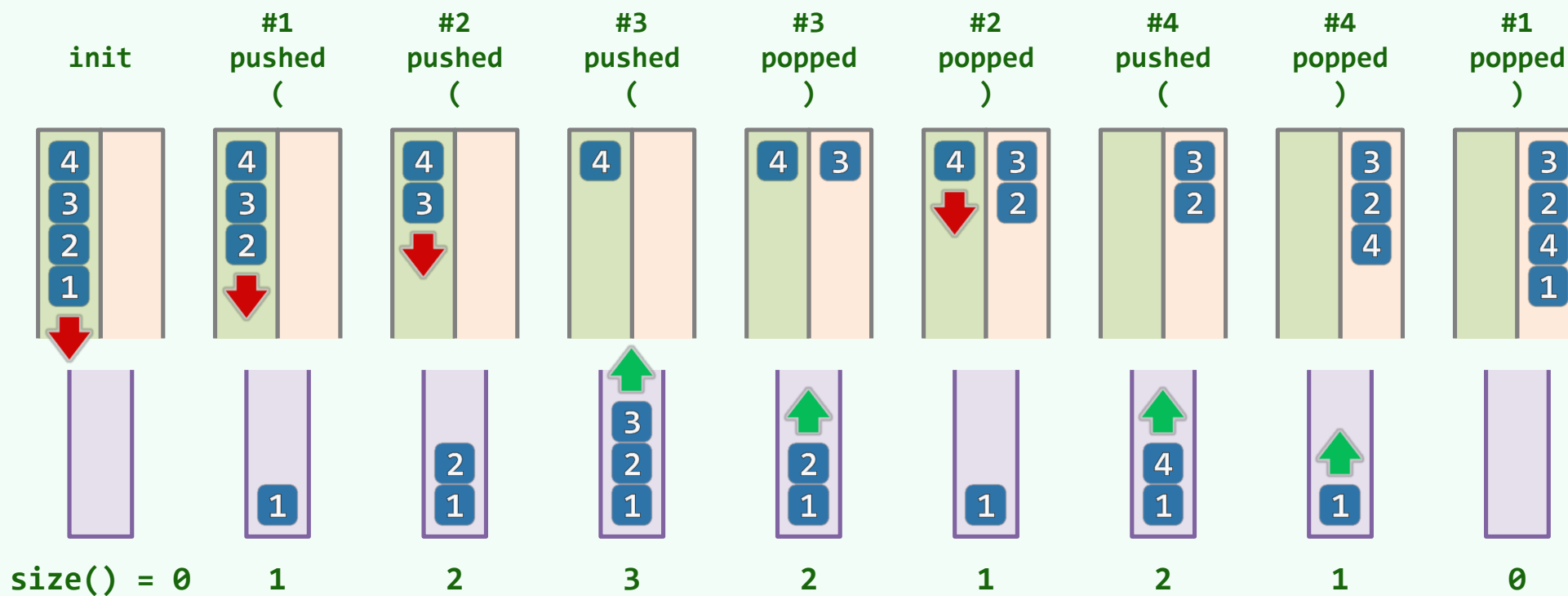
❖ 如此，可得一个 $O(n^2)$ 的甄别算法 //再进一步地...

❖ $O(n)$ 算法：直接借助栈A、B和S，模拟混洗过程 //为何可行？

每次S.pop()之前，检测S是否已空；或需弹出的元素在S中，却非顶元素

括号匹配

❖ 观察：每一栈混洗，都对应于栈S的n次push与n次pop操作构成的某一序列；反之亦然



❖ n个元素的栈混洗，等价于n对括号的匹配；二者的组合数，也自然相等

栈

中缀表达式求值：问题与构思

应用

❖ 给定语法正确的算术表达式s，计算与之对应的数值

❖ UNIX: `$ echo $(( 0 + ( 1 + 23 ) / 4 * 5 * 67 - 8 + 9 ))`

❖ DOS: `\> set /a ( !0 ^<^< ( 1 - 2 + 3 * 4 ) ) - 5 * ( 6 ^| 7 ) / ( 8 ^^ 9 )`

❖ PS: `GS> 0 1 23 add 4 div 5 mul 67 mul add 8 sub 9 add =`

❖ Excel: `= COS(0) + 1 - ( 2 - POWER( ( FACT(3) - 4 ), 5) ) * 67 - 8 + 9`

❖ Word: `= NOT(0) + 12 + 34 * 56 + 7 + 89`

❖ calc: `0 ! + 12 + 34 * 56 + 7 + 89 =`

❖ calc: `0 ! + 1 - ( 2 - ( 3 ! - 4 ) y 5 ) * 67 - 8 + 9 =`

减而治之

❖ 优先级高的**局部**执行计算，并被代以其**数值**

运算符渐少，直至得到最终结果

1 4 8 1

1 4 5 8 + 2 3

2 × 7 2 9 + 2 3

2 × 3 ^ 6 + 2 3

2 × 3 ^ (2 × 3) + 2 3

❖ $\text{str}(v)$: 数值 v 对应的字符串 (名)

$\text{val}(S)$: 符号串 S 对应的数值 (实)

❖ 设表达式: $S = S_L + S_0 + S_R$

- S_0 可优先计算, 且

- $\text{val}(S_0) = v_0$

❖ 则有递推化简关系

$\text{val}(S) = \text{val}(S_L + \text{str}(v_0) + S_R)$

优先级

❖ 难点：如何高效地**找到**可优先计算的 s_0 。

(亦即，其对应的运算符)？

1 4 8 1

1 4 5 8 + 2 3

2 × 7 2 9 + 2 3

2 × 3 ^ 6 + 2 3

2 × 3 ^ (2 × 3) + 2 3

❖ 与**括号匹配**迭代版类似，但亦不尽相同

- 不能简单地按“左先右后”次序处理各运算符
- 此时，需要考虑更多因素...

❖ 约定俗成的**优先级**：

$1 + 2 * 3 ^ 4 !$

可强行改变次序的**括号**：

$(((1 + 2) * 3) ^ 4) !$

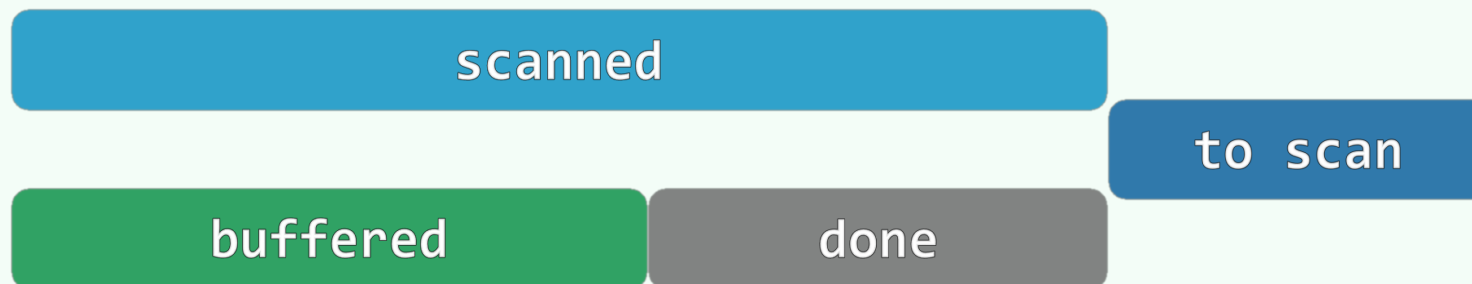
延迟缓冲

❖ 仅根据表达式的前缀，不足以确定各运算符的计算次序

只有获得足够的后续信息，才能确定其中哪些运算符可以执行

❖ 体现在求值算法的流程上

为处理某一前缀，必须提前预读并分析更长的前缀



❖ 为此，需借助某种支持延迟缓冲的机制...

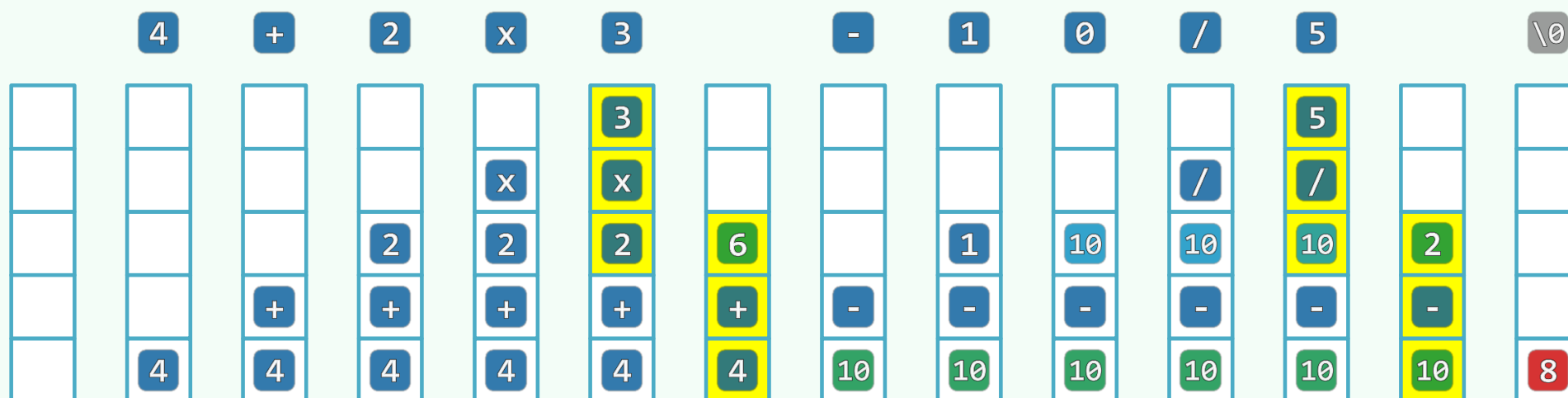
求值算法 = 栈 + 线性扫描

❖ 自左向右扫描表达式，用栈记录已扫描的部分（含已执行运算的结果）

栈的**顶部**存在**可优先计算**的子表达式

？ 该子表达式退栈；计算其数值；计算结果进栈

： 当前字符进栈，转入下一字符



❖ 只要语法正确，则栈内最终应只剩一个元素 //即表达式对应的数值

栈
中缀表达式求值： 算法

主算法

```
❖ double evaluate( char* S, char* RPN ) { //S保证语法正确
    Stack<double> opnd; Stack<char> optr; //运算数栈、运算符栈
    optr.push( '\0' ); //铺垫
    while ( ! optr.empty() ) { //逐个处理各字符，直至运算符栈空
        if ( isdigit( *S ) ) //若为操作数（可能多位、小数），则
            readNumber( S, opnd ); //读入
        else //若为运算符，则视其与栈顶运算符之间优先级的高低
            switch( priority( optr.top(), *S ) ) { /* 分别处理 */ }
    } //while
    return opnd.pop(); //弹出并返回最后的计算结果
}
```

优先级表

```
const char pri[N_OPTR][N_OPTR] = { //运算符优先等级 [栈顶][当前]
    /* -- + */ '>', '>', '<', '<', '<', '<', '<', '>', '>',
    /* | - */ '>', '>', '<', '<', '<', '<', '<', '>', '>',
    /* 栈 * */ '>', '>', '>', '>', '<', '<', '<', '>', '>',
    /* 顶 / */ '>', '>', '>', '>', '<', '<', '<', '>', '>',
    /* 运 ^ */ '>', '>', '>', '>', '>', '<', '<', '>', '>',
    /* 算 ! */ '>', '>', '>', '>', '>', '>', ' ', '>', '>',
    /* 符 ( */ '<', '<', '<', '<', '<', '<', '<', '=', ' ',
    /* | ) */ ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ',
    /* -- \0 */ '<', '<', '<', '<', '<', '<', '<', ' ', '=',
    //      +      -      *      /      ^      !      (      )      \0
    //      |----- 当前运算符 -----|
};
```

'<': 静待时机: 算法

❖ `switch( priority( optr.top(), *S ) ) {`

`case '<': //栈顶运算符优先级更低`

`optr.push( *S ); S++; break; //计算推迟, 当前运算符进栈`

`case '=':`

`/* ..... */`

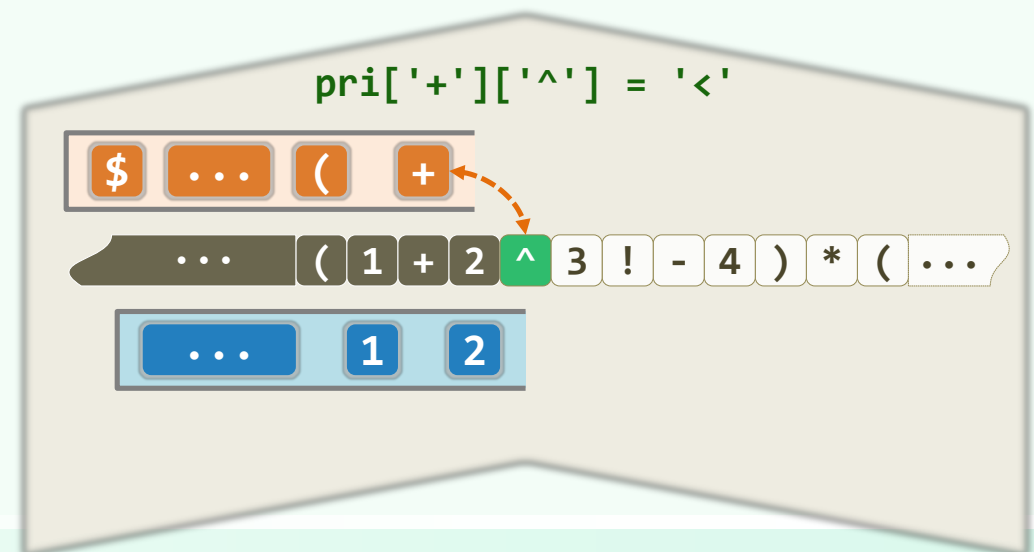
`case '>': {`

`/* ..... */`

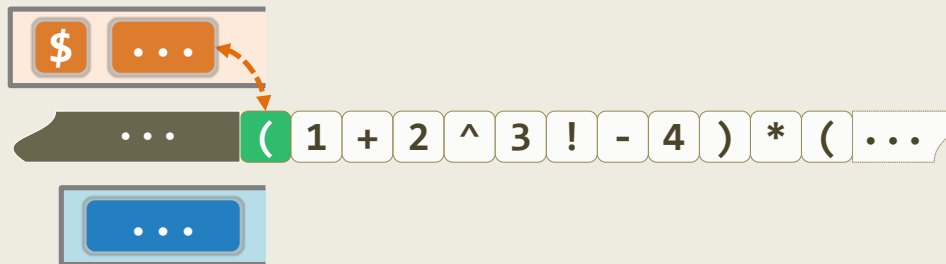
`break;`

`} //case '>'`

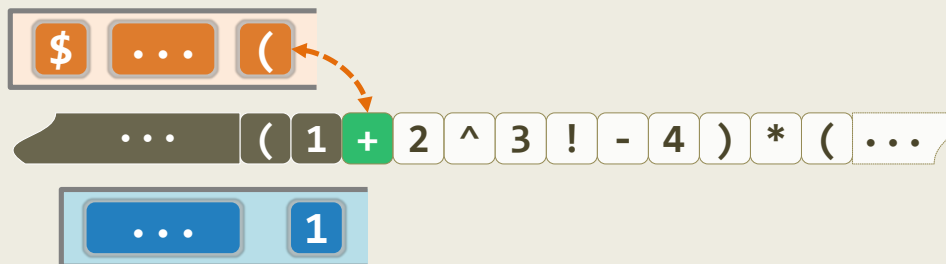
`} //switch`



'<': 静待时机: 实例

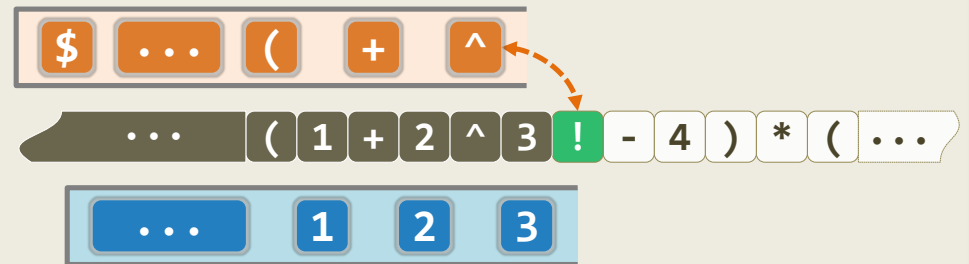


`pri[ ][ '(' ] = '<'`

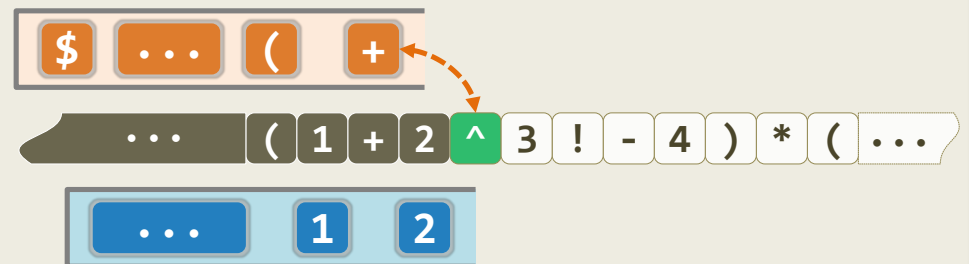


`pri['('][ '+' ] = '<'`

`pri['^']['!'] = '<'`



`pri['+']['^'] = '<'`



'>': 时机已到: 算法

```
❖ switch( priority( optr.top(), *S ) ) {
```

```
    /* ..... */
```

```
    case '>': {
```

```
        char op = optr.pop();
```

```
        if ( '!' == op ) opnd.push( calcu( op, opnd.pop() ) ); //一元运算符
```

```
        else { double pOpnd2 = opnd.pop(), pOpnd1 = opnd.pop(); //二元运算符
```

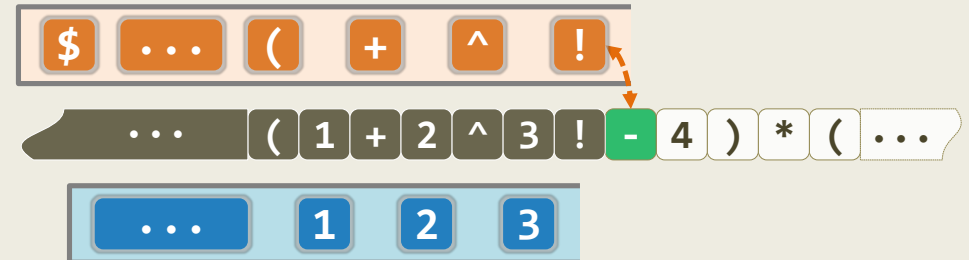
```
                opnd.push( calcu( pOpnd1, op, pOpnd2 ) ); //实施计算, 结果入栈
```

```
        } //为何不直接: opnd.push( calcu( opnd.pop(), op, opnd.pop() ) ) ?
```

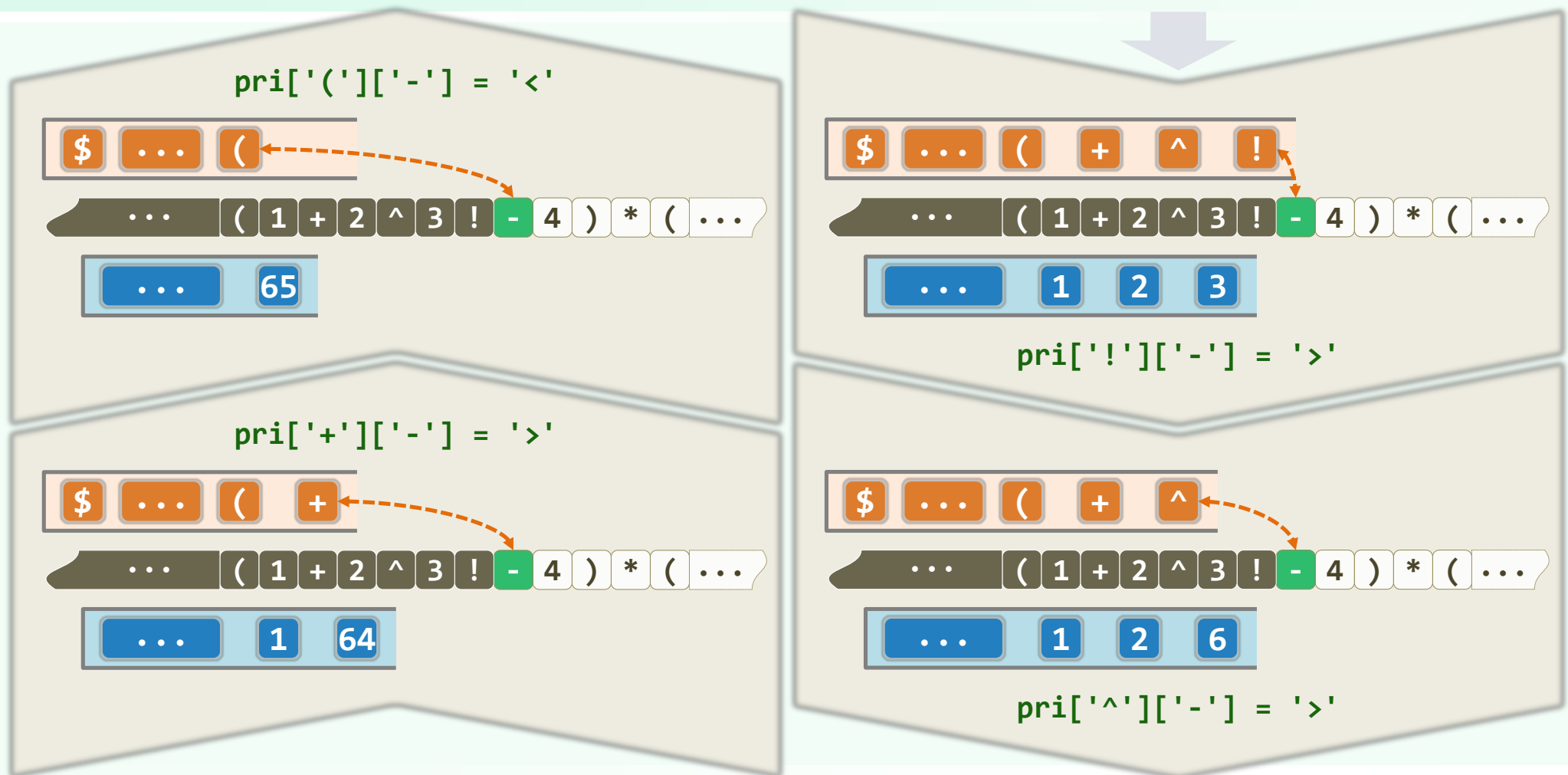
```
        break;
```

```
    } //case '>'
```

```
} //switch
```



'>': 时机已到: 实例



'='：终须了断：算法

❖ `switch( priority( optr.top(), *S ) ) {`

`case '<':`

`/* ..... */`

`case '=': //优先级相等（当前运算符为右括号，或尾部哨兵'\0'）`

`optr.pop(); S++; break; //脱括号并接收下一个字符`

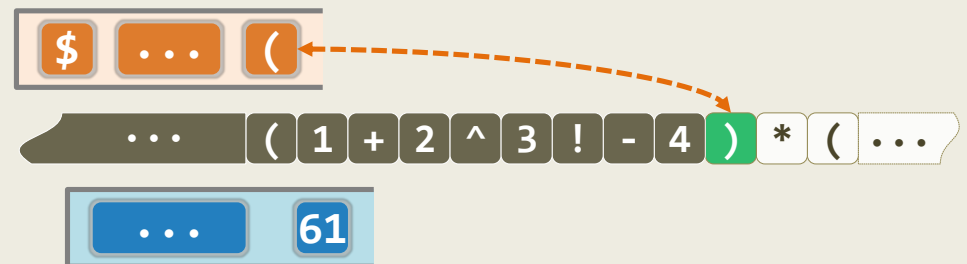
`case '>': {`

`/* ..... */`

`break;`

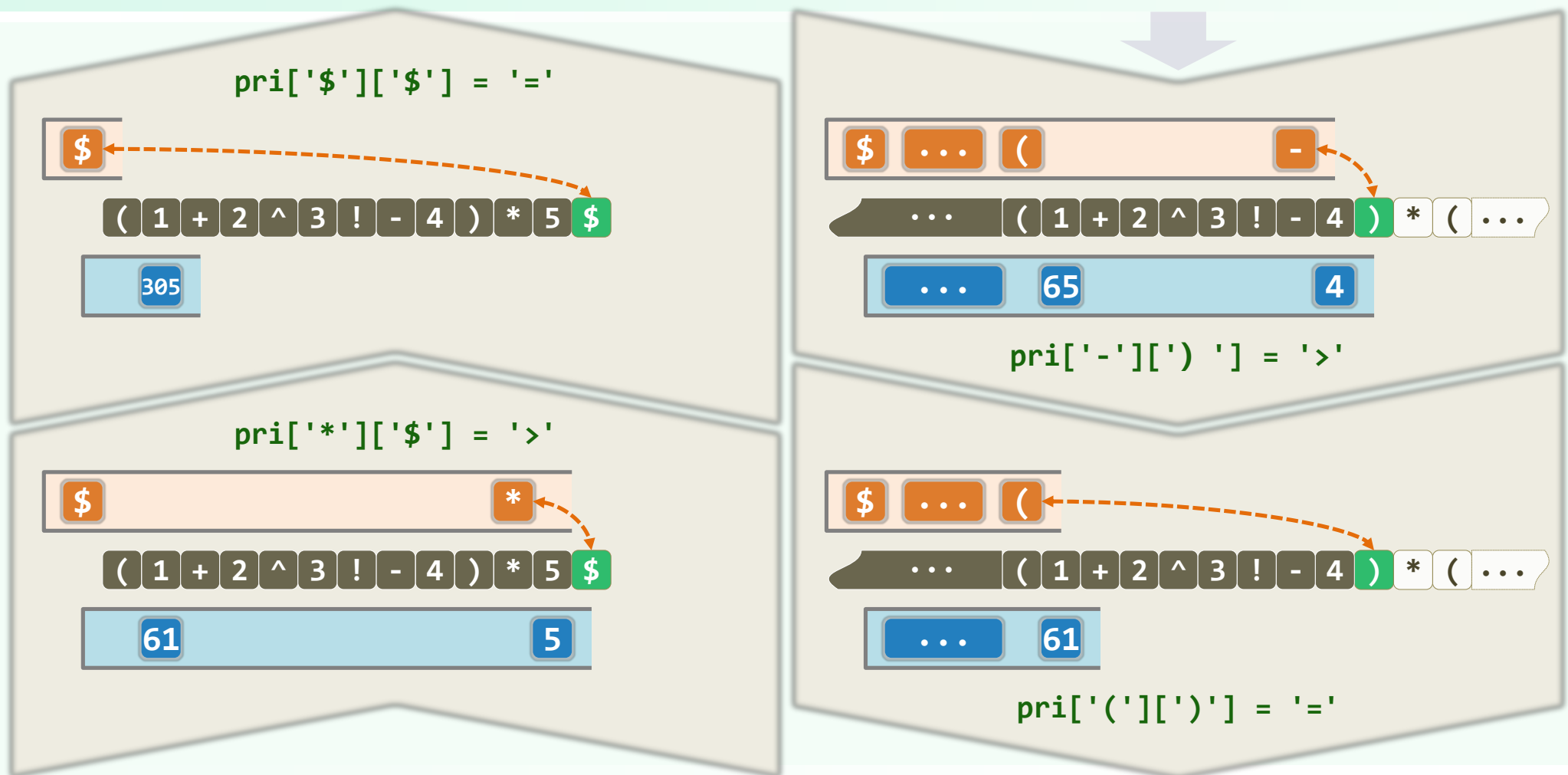
`} //case '>'`

`} //switch`



`pri['('][')'] = '='`

'=': 终须了断: 实例



+ - * / !: 芸芸众生

const char pri[N_OPTR][N_OPTR] = { //运算符优先等级 [栈顶][当前]

/* -- + */	'>', '>',	'<', '<', '<', '<',	'<', '>', '>',						
/* - */	'>', '>',	'<', '<', '<', '<',	'<', '>', '>',						
/* 栈 * */	'>', '>',	'>', '>',	'<', '<', '<', '>', '>',						
/* 顶 / */	'>', '>',	'>', '>',	'<', '<', '<', '>', '>',						
/* 运 ^ */	'>', '>',	'>', '>',	'>', '<', '<', '>', '>',						
/* 算 ! */	'>', '>',	'>', '>',	'>', '>', '>', '>',						
/* 符 (*/	'<', '<', '<', '<', '<', '<',	'<', '<', '<', '<', '<', '<',	'<', '<', '<', '<', '<', '<',						
/*) */	'<', '<', '<', '<', '<', '<',	'<', '<', '<', '<', '<', '<',	'<', '<', '<', '<', '<', '<',						
/* -- \0 */	'<', '<', '<', '<', '<', '<',	'<', '<', '<', '<', '<', '<',	'<', '<', '<', '<', '<', '<',						
//	+	-	*	/	^	!	(	)	\0
//	----- 当前运算符 -----								

};

'(': 我不下地狱，谁下地狱

```

const char pri[N_OPTR][N_OPTR] = { //运算符优先级 [栈顶][当前]

/* -- + */      '>', '>', '<', '<', '<', '<', '<', '>', '>',
/* | - */      '>', '>', '<', '<', '<', '<', '<', '>', '>',
/* 栈 * */      '>', '>', '>', '>', '<', '<', '<', '>', '>',
/* 顶 / */      '>', '>', '>', '>', '<', '<', '<', '>', '>',
/* 运 ^ */      '>', '>', '>', '>', '>', '<', '<', '>', '>',
/* 算 ! */      '>', '>', '>', '>', '>', '>', '>', '>', '>',
/* 符 ( */      '<', '<', '<', '<', '<', '<', '<', '=', ' ',
/* | ) */      ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ',
/* -- \0 */      '<', '<', '<', '<', '<', '<', '<', ' ', '=',

//          +      -      *      /      ^      !      (      )      \0
//          |----- 当前运算符 -----|
};

```

Data Structures & Algorithms, Tsinghua University

') ' : 死线已至 (然后满血复活)

const char pri[N_OPTR][N_OPTR] = { //运算符优先等级 [栈顶][当前]

/* -- + */	'>', '>', '<', '<', '<', '<', '<', '>', '>',
/* - */	'>', '>', '<', '<', '<', '<', '<', '>', '>',
/* 栈 * */	'>', '>', '>', '>', '<', '<', '<', '>', '>',
/* 顶 / */	'>', '>', '>', '>', '<', '<', '<', '>', '>',
/* 运 ^ */	'>', '>', '>', '>', '>', '<', '<', '>', '>',
/* 算 ! */	'>', '>', '>', '>', '>', '>', '>', '>', '>',
/* 符 (*/	'<', '<', '<', '<', '<', '<', '<', '=',
/*) */	' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ',
/* -- \0 */	'<', '<', '<', '<', '<', '<', '<', ' ', '=',
//	+ - * / ^ ! () \0
//	----- 当前运算符 -----

};

'\0': 从创世纪，到世界末日

```
const char pri[N_OPTR][N_OPTR] = { //运算符优先等级 [栈顶][当前]
```

Diagram illustrating the evaluation of the postfix expression `1 2 3 4 5 + ^ * /`. The expression is shown in a grid of boxes. The current operator being evaluated is `/`, which is highlighted in red. The stack contains the values 1, 2, 3, 4, and 5. The result of the previous operation (`5 + 4 = 9`) is shown in the stack. The current operator `/` is being applied to the top two elements of the stack (9 and 3). The result of this operation (`9 / 3 = 3`) is shown in the stack. The final result of the expression is 3.

 $\}:$

栈 中缀表达式求值：实例

表达式	运算符栈	操作数栈	注解
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$		表达式起始标识入栈
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ (		左括号入栈
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ (	0	操作数0入栈
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ (!	0	运算符'!'入栈
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ (	1	运算符'!'出栈执行
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ (+	1	运算符'+'入栈
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ (+	1 1	操作数1入栈
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ (	2	运算符'+'出栈执行
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$	2	左括号出栈
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ *	2	运算符'*'入栈
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ *	2 2	操作数2入栈

表达式	运算符栈	操作数栈	注解
$(0!+1)*2^{\wedge}(3!+4)-(5!-67-(8+9))\$$	\$ * ^	2 2	运算符'^'入栈
$(0!+1)*2^{\wedge}(3!+4)-(5!-67-(8+9))\$$	\$ * ^ (	2 2	左括号入栈
$(0!+1)*2^{\wedge}(3!+4)-(5!-67-(8+9))\$$	\$ * ^ (	2 2 3	操作数3入栈
$(0!+1)*2^{\wedge}(3!+4)-(5!-67-(8+9))\$$	\$ * ^ (!	2 2 3	运算符'!'入栈
$(0!+1)*2^{\wedge}(3!+4)-(5!-67-(8+9))\$$	\$ * ^ (	2 2 6	运算符'!'出栈执行
$(0!+1)*2^{\wedge}(3!+4)-(5!-67-(8+9))\$$	\$ * ^ (+	2 2 6	运算符'+'入栈
$(0!+1)*2^{\wedge}(3!+4)-(5!-67-(8+9))\$$	\$ * ^ (+	2 2 6 4	操作数4入栈
$(0!+1)*2^{\wedge}(3!+4)-(5!-67-(8+9))\$$	\$ * ^ (	2 2 10	运算符'+'出栈执行
$(0!+1)*2^{\wedge}(3!+4)-(5!-67-(8+9))\$$	\$ * ^	2 2 10	左括号出栈
$(0!+1)*2^{\wedge}(3!+4)-(5!-67-(8+9))\$$	\$ *	2 1024	运算符'^'出栈执行
$(0!+1)*2^{\wedge}(3!+4)-(5!-67-(8+9))\$$	\$	2048	运算符'*'出栈执行

表达式	运算符栈	操作数栈	注解
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ -	2048	运算符'-'入栈
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ - (	2048	左括号入栈
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ - (	2048 5	操作数5入栈
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ - (!	2048 5	运算符'!'入栈
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ - (	2048 120	运算符'!'出栈执行
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ - (-	2048 120	运算符'-'入栈
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ - (-	2048 120 67	操作数67入栈
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ - (	2048 53	运算符'-'出栈执行
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ - (-	2048 53	运算符'-'入栈
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ - (- (	2048 53	左括号入栈
$(0!+1)*2^{(3!+4)}-(5!-67-(8+9))\$$	\$ - (- (	2048 53 8	操作数8入栈

表达式	运算符栈	操作数栈	注解
$(0!+1)*2^{\wedge}(3!+4)-(5!-67-(8+9))\$$	\$ - (- (+	2048 53 8	运算符 '+' 入栈
$(0!+1)*2^{\wedge}(3!+4)-(5!-67-(8+9))\$$	\$ - (- (+	2048 53 8 9	操作数 9 入栈
$(0!+1)*2^{\wedge}(3!+4)-(5!-67-(8+9))\$$	\$ - (- (	2048 53 17	运算符 '+' 出栈执行
$(0!+1)*2^{\wedge}(3!+4)-(5!-67-(8+9))\$$	\$ - (-	2048 53 17	左括号出栈
$(0!+1)*2^{\wedge}(3!+4)-(5!-67-(8+9))\$$	\$ - (	2048 36	运算符 '-' 出栈执行
$(0!+1)*2^{\wedge}(3!+4)-(5!-67-(8+9))\$$	\$ -	2048 36	左括号出栈
$(0!+1)*2^{\wedge}(3!+4)-(5!-67-(8+9))\$$	\$	2012	运算符 '-' 出栈执行
$(0!+1)*2^{\wedge}(3!+4)-(5!-67-(8+9))\$$		2012	表达式起始标识出栈
$(0!+1)*2^{\wedge}(3!+4)-(5!-67-(8+9))\$$			返回唯一的元素 2012

栈

逆波兰表达式：定义与求值

Reverse Polish Notation

- ❖ 逆波兰表达式: J. Lukasiewicz (1878 ~ 1956)
- ❖ 在由运算符 (operator) 和操作数 (operand) 组成的表达式中
不使用括号 (parenthesis-free), 即可表示带优先级的运算关系
- ❖ 例如: $0 ! + 123 + 4 * (5 * 6 ! + 7 ! / 8) / 9$
 $0 ! 123 + 4 5 6 ! * 7 ! 8 / + * 9 / +$
- ❖ 又如: $(0 ! + 1) ^ (2 * 3 ! + 4 - 5) - 6 ! / (7 + 8 + 9)$
 $0 ! 1 + 2 3 ! * 4 + 5 - ^ 6 ! 7 8 + 9 + / -$
- ❖ 相对于日常使用的中缀式 (infix), RPN亦称作后缀式 (postfix)
- ❖ 作为补偿, 须额外引入一个起分隔作用的**元字符** (比如空格) //较之原表达式, 未必更短

栈式求值

0 ! 123 + 4 5 6 ! * 7 ! 8 / + * 9 / +

❖ 引入栈s //用以存放操作数

逐个处理下一元素x

if (x是操作数) 将x压入s

else //x是运算符 (无需缓冲)

从s中弹出x所需数目的操作数

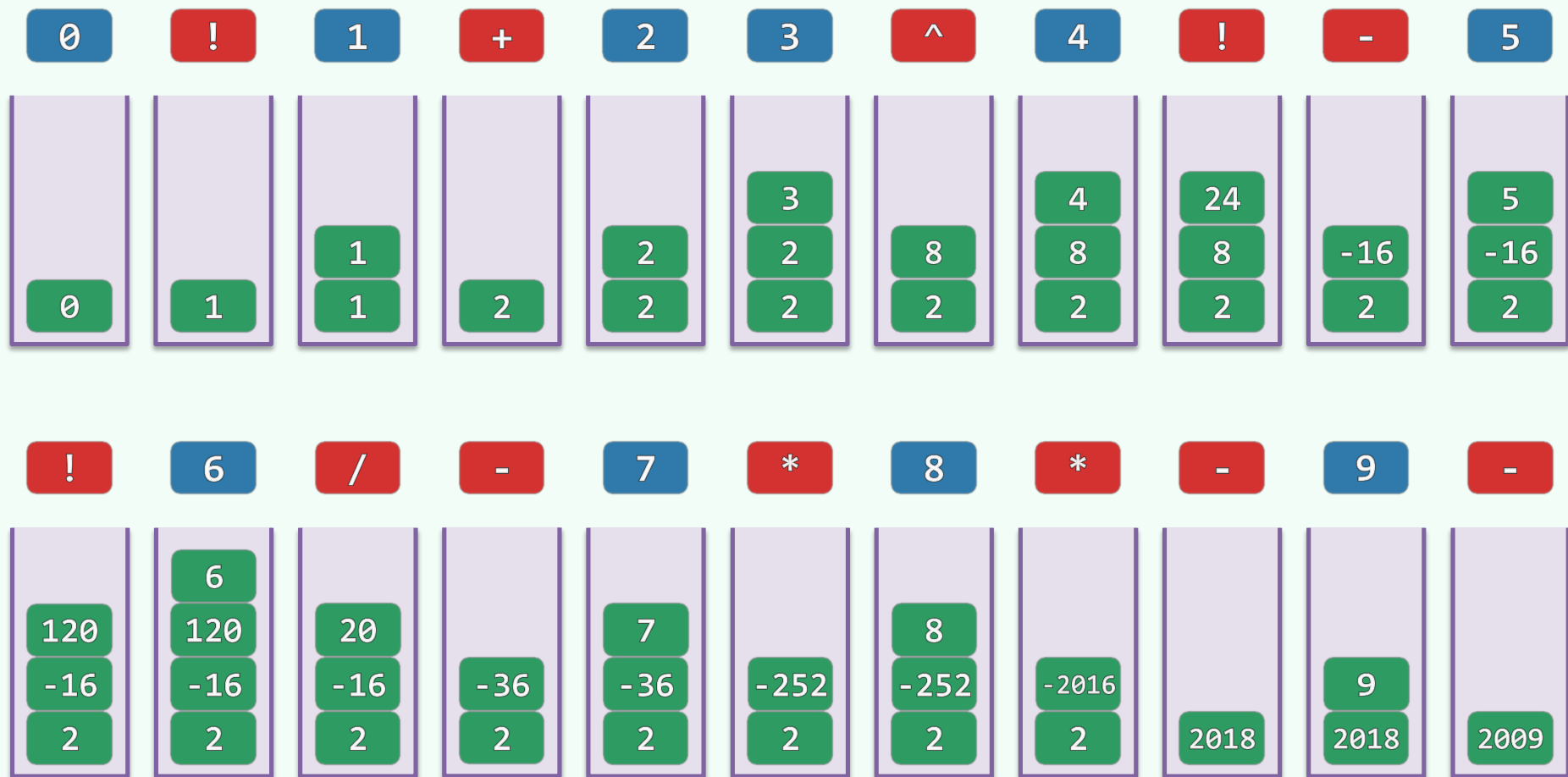
执行相应的计算, 结果压入s //无需顾及优先级!

返回栈顶

❖ 只要输入的RPN语法正确, 此时的栈顶亦是栈底, 对应于最终的计算结果

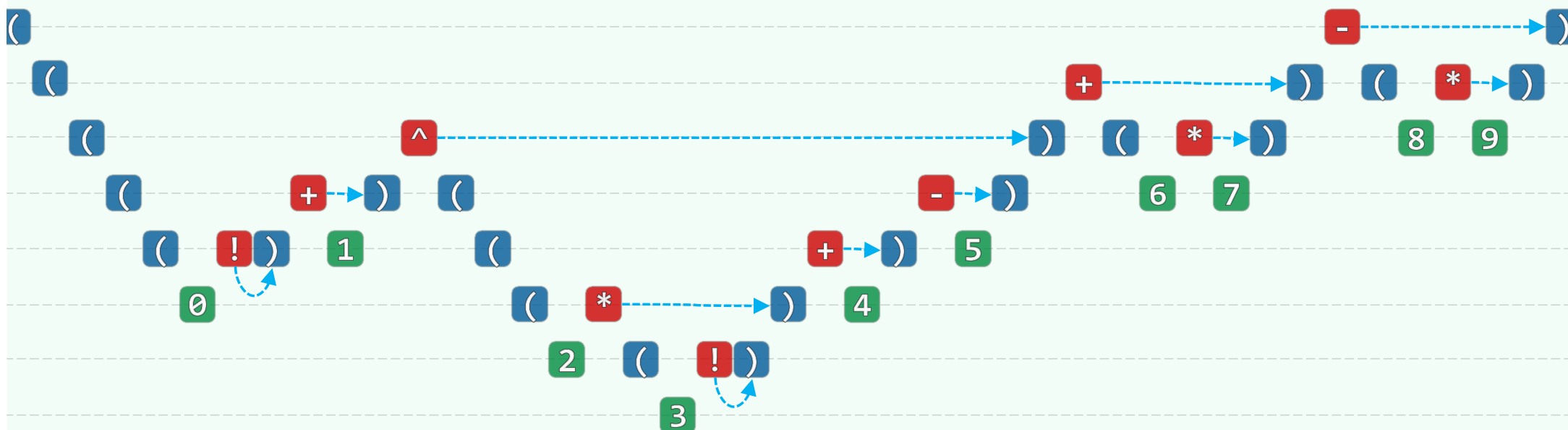
630
4230
1880
2004

0 ! 1 + 2 3 ^ 4 ! - 5 ! 6 / - 7 * 8 * - 9 -



栈
逆波兰表达式： 转换

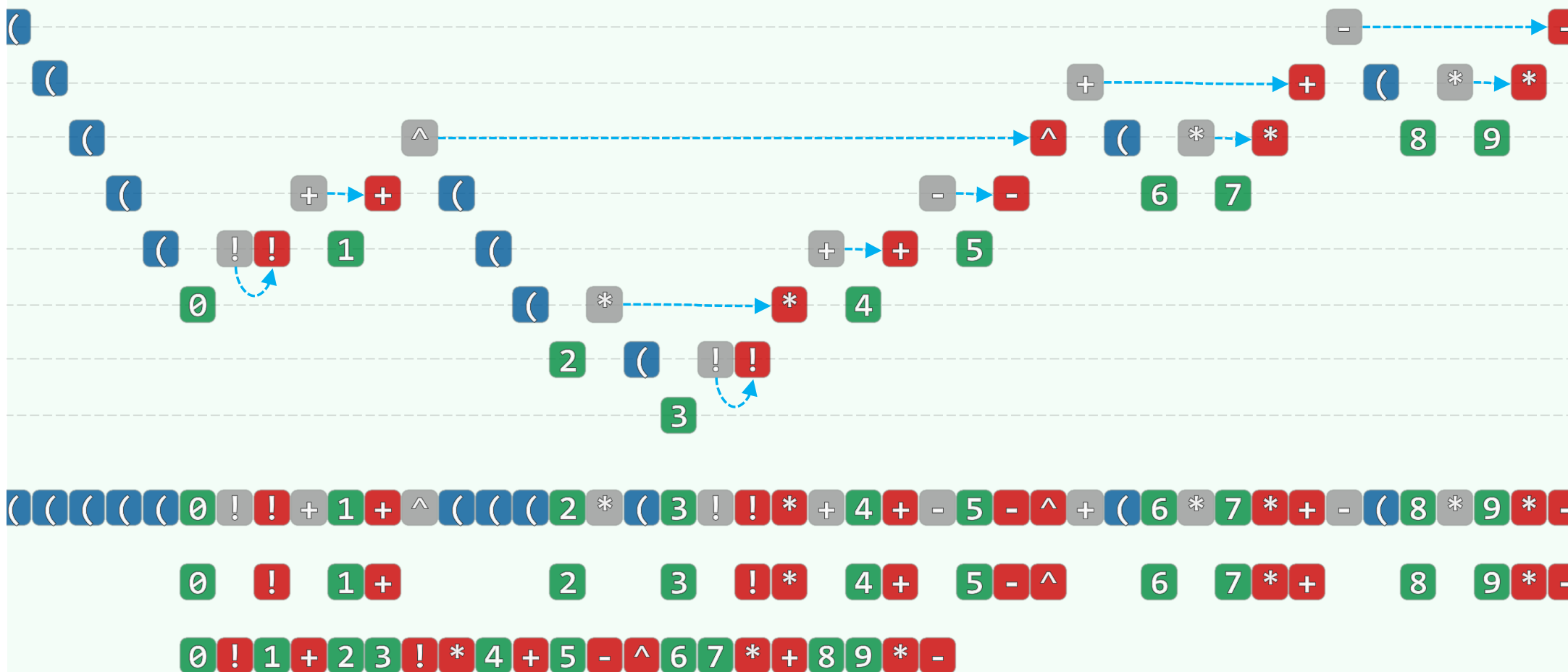
手工转换：添加括号



(((((0!)+1)^(((2*((3!)+4)-5))+((6*7))-((8*9))))))

((0!+1)^((2*3!+4-5)+6*7-8*9))

手工转换：以运算符替换右括号，清除左括号



自动转换

```
double evaluate( char* S, char* RPN ) { //RPN转换
    /* ..... */
    while ( ! optr.empty() ) { //逐个处理各字符, 直至运算符栈空
        if ( isdigit( * S ) ) //若当前字符为操作数, 则直接
            { readNumber( S, opnd ); append( RPN, opnd.top() ); } //将其接入RPN
        else //若当前字符为运算符
            switch( priority( optr.top(), *S ) ) {
                /* ..... */
                case '>': { //且可立即执行, 则在执行相应计算的同时
                    char op = optr.pop(); append( RPN, op ); //将其接入RPN
                    /* ..... */
                } //case '>'

                /* ..... */
            }
    }
}
```